Python 程式語言

Lecture 6: Object-Oriented Programming

Instructor: 顏家珩 Chia-Heng Yen

Course Roadmap

- Fundamentals & Environment
  - Introduction to Python and Development Environment Setup
  - Basic Syntax and Flow Control
  - Data Structures and Collections
- Advanced Programming
  - Modular and Functional Programming
  - File Handling and Exception Handling
  - Object-Oriented Programming
  - Algorithm Implementation in Python
  - Modules and Package Applications
- User Interface Development
  - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
  - Data Analysis Fundamentals
  - Scientific Computing Applications
  - Data Visualization
- AI & ML Applications
  - Introduction to Machine Learning with Applications
  - AI-Related Library in Python Applications

Outline

- Learn object-oriented programming basics
- Create classes and objects
- Implement inheritance and polymorphism

What is OOP?

- Definition
  - Object-Oriented Programming (OOP) is a programming paradigm based on objects that contain data and methods
- Four Pillars of OOP
  - Encapsulation (封裝): Bundling data and methods together
  - Abstraction (抽象): Hiding complex implementation details
  - Inheritance (繼承): Creating new classes from existing ones
  - Polymorphism (多型): Using a single interface for different types
- Why OOP?
  - Code Reusability
  - Modularity
  - Easier Maintenance
  - Better Organization
  - Real-world Modeling

Classes and Objects

- Basic Class Definition

```python
# Define a class
class Dog:
    """A simple Dog class"""

    # Class attribute (shared by all instances)
    species = "Canis familiaris"

    # Constructor (initializer)
    def __init__(self, name, age):
        # Instance attributes (unique to each instance)
        self.name = name
        self.age = age

    # Instance method
    def bark(self):
        return f"{self.name} says Woof!"
    def description(self):
        return f"{self.name} is {self.age} years old"
```

```python
# Create objects (instances)
dog1 = Dog("Buddy", 3)
dog2 = Dog("Max", 5)

# Access attributes
print(dog1.name) # Buddy
print(dog1.age) # 3
print(dog1.species) # Canis familiaris

# Call methods
print(dog1.bark()) # Buddy says Woof!
print(dog1.description()) # Buddy is 3 years old

# Each object is independent
print(dog2.name) # Max
print(dog2.bark()) # Max says Woof!
```

Classes and Objects

- Class vs Instance

```python
class Person:
    """Person class demonstrating class vs instance attributes"""

    # Class attribute
    population = 0

    def __init__(self, name, age):
        # Instance attributes
        self.name = name
        self.age = age

        # Modify class attribute
        Person.population += 1

    def introduce(self):
        return f"Hi, I'm {self.name}, {self.age} years old"

    @classmethod
    def get_population(cls):
        return f"Total population: {cls.population}"
```

```python
# Create instances
person1 = Person("Alice", 25)
person2 = Person("Bob", 30)
person3 = Person("Carol", 28)

# Access instance attributes
print(person1.name) # Alice
print(person2.name) # Bob

# Access class attribute
print(Person.population) # 3
print(person1.population) # 3 (can access via instance too)
print(Person.get_population()) # Total population: 3
```

Constructor and Destructor

- __init__ Constructor

```python
class Book:
    """Book class with constructor"""

    def __init__(self, title, author, pages, price=0):
        """Initialize a new book"""
        print(f"Creating book: {title}")

        # Instance attributes
        self.title = title
        self.author = author
        self.pages = pages
        self.price = price
        self.is_available = True

    def display_info(self):
        """Display book information"""
        print(f"Title: {self.title}")
        print(f"Author: {self.author}")
        print(f"Pages: {self.pages}")
        print(f"Price: ${self.price}")
        print(f"Available: {self.is_available}")

# Create book objects
book1 = Book("Python Programming", "John Doe", 500, 49.99)
book2 = Book("Data Science", "Jane Smith", 400)
```

```python
# Display information
book1.display_info()
print("-" * 40)
book2.display_info()

# Constructor with validation
class Student:
    """Student class with input validation"""
    def __init__(self, name, age, student_id):
        """Initialize student with validation"""
        # Validate inputs
        if not name:
            raise ValueError("Name cannot be empty")
        if age < 0 or age > 150:
            raise ValueError("Invalid age")
        if not isinstance(student_id, str) or len(student_id) != 8:
            raise ValueError("Student ID must be 8 characters")

        # Set attributes
        self.name = name
        self.age = age
        self.student_id = student_id
        self.courses = []
```

```python
    def enroll(self, course):
        """Enroll in a course"""
        self.courses.append(course)
        print(f"{self.name} enrolled in {course}")

# Test student creation
try:
    student1 = Student("Alice", 20, "20240001")
    student1.enroll("Python Programming")

    # This will fail validation
    student2 = Student("", 20, "20240002")
except ValueError as e:
    print(f"Error: {e}")
```

Constructor and Destructor

- __del__ Destructor

```
class FileHandler:
    """Class demonstrating destructor"""

    def __init__(self, filename):
        """Initialize file handler"""
        self.filename = filename
        print(f"Opening file: {filename}")
        # In real code, you would open the file here

    def __del__(self):
        """Destructor - called when object is destroyed"""
        print(f"Closing file: {self.filename}")
        # In real code, you would close the file here

 Create and destroy object
handler = FileHandler("data.txt")
print("Working with file...")
del handler # Explicitly delete object
print("File handler deleted")
```

```
# Destructor called automatically
def test_destructor():
    handler = FileHandler("temp.txt")
    print("Inside function")
    # Object destroyed when function ends

test_destructor()
print("Function completed")

# Note: __del__ is rarely needed in Python
# Use context managers (with statement) instead
```

Instance, Class, and Static Methods

- Instance Methods

```
class Calculator:
    """Calculator with instance methods"""

    def __init__(self, name):
        self.name = name
        self.result = 0

    # Instance method - works with instance data
    def add(self, value):
        """Add to current result"""
        self.result += value
        return self.result

    def subtract(self, value):
        """Subtract from current result"""
        self.result -= value
        return self.result
```

```
    def reset(self):
        """Reset calculator"""
        self.result = 0

    def get_result(self):
        """Get current result"""
        return self.result

# Use instance methods
calc = Calculator("My Calculator")
print(calc.add(10)) # 10
print(calc.add(5)) # 15
print(calc.subtract(3)) # 12
print(calc.get_result()) # 12
calc.reset()
print(calc.get_result()) # 0
```

Instance, Class, and Static Methods

- Class Methods

```
class Employee:
    """Employee class with class methods"""
    # Class attributes
    company_name = "TechCorp"
    employee_count = 0
    min_salary = 30000
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
        Employee.employee_count += 1

    # Instance method
    def display_info(self):
        return f"{self.name} - ${self.salary}"

    # Class method - works with class data
    @classmethod
    def get_employee_count(cls):
        """Get total number of employees"""
        return cls.employee_count
    @classmethod
    def set_company_name(cls, name):
        """Set company name"""
        cls.company_name = name
    @classmethod
    def get_min_salary(cls):
        """Get minimum salary"""
        return cls.min_salary

    # Class method as alternative constructor
    @classmethod
    def from_string(cls, emp_string):
        """Create employee from string: 'Name,Salary'"""
        name, salary = emp_string.split(',')
        return cls(name, int(salary))
# Use class methods
emp1 = Employee("Alice", 50000)
emp2 = Employee("Bob", 60000)

print(Employee.get_employee_count()) # 2
print(Employee.get_min_salary()) # 30000

# Alternative constructor
emp3 = Employee.from_string("Carol,55000")
print(emp3.display_info()) # Carol - $55000

# Change class data
Employee.set_company_name("NewTech Inc.")
print(Employee.company_name) # NewTech Inc.
```

Instance, Class, and Static Methods

- Static Methods

```
class MathOperations:
    """Math operations with static methods"""

    # Static method - doesn't access instance or class data
    @staticmethod
    def add(a, b):
        """Add two numbers"""
        return a + b

    @staticmethod
    def multiply(a, b):
        """Multiply two numbers"""
        return a * b

    @staticmethod
    def is_even(number):
        """Check if number is even"""
        return number % 2 == 0

    @staticmethod
    def factorial(n):
        """Calculate factorial"""
        if n <= 1:
            return 1
        return n * MathOperations.factorial(n - 1)
```

```
# Use static methods
# Can call without creating instance
print(MathOperations.add(5, 3)) # 8
print(MathOperations.multiply(4, 7)) # 28
print(MathOperations.is_even(10)) # True
print(MathOperations.factorial(5)) # 120

# Can also call via instance (but not common)
math_ops = MathOperations()
print(math_ops.add(2, 3)) # 5
```

Instance, Class, and Static Methods

- Comparison of method types

```
class Demo:
    class_var = "Class Variable"

    def __init__(self, value):
        self.instance_var = value

    # Instance method: has access to self (instance)
    def instance_method(self):
        return f"Instance: {self.instance_var}, Class: {self.class_var}"

    # Class method: has access to cls (class)
    @classmethod
    def class_method(cls):
        return f"Class: {cls.class_var}"

    # Static method: no access to self or cls
    @staticmethod
    def static_method(param):
        return f"Static: {param}"
```

```
# Test different method types
obj = Demo("Instance Value")
print(obj.instance_method()) # Needs instance
print(Demo.class_method()) # Works on class
print(Demo.static_method("Test")) # Independent
```

Inheritance

- Basic Inheritance

```python
# Parent class (Base class, Superclass)
class Animal:
    """Base Animal class"""
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def make_sound(self):
        return "Some generic sound"
    def info(self):
        return f"{self.name} is {self.age} years old"

# Child class (Derived class, Subclass)
class Dog(Animal):
    """Dog class inheriting from Animal"""
    def __init__(self, name, age, breed):
        # Call parent constructor
        super().__init__(name, age)
        self.breed = breed
    # Override parent method
    def make_sound(self):
        return "Woof! Woof!"
    # Add new method
    def fetch(self):
        return f"{self.name} is fetching the ball!"
```

```python
class Cat(Animal):
    """Cat class inheriting from Animal"""
    def __init__(self, name, age, color):
        super().__init__(name, age)
        self.color = color
    def make_sound(self):
        return "Meow!"
    def scratch(self):
        return f"{self.name} is scratching!"

# Create instances
dog = Dog("Buddy", 3, "Golden Retriever")
cat = Cat("Whiskers", 2, "Gray")

# Use inherited methods
print(dog.info()) # From Animal class
print(dog.make_sound()) # Overridden in Dog class
 print(dog.fetch()) # Dog-specific method Dog

print(cat.info()) # From Animal class
print(cat.make_sound()) # Overridden in Cat class
print(cat.scratch()) # Cat-specific method
```

Inheritance

- Multiple Inheritance

```
# Multiple parent classes
class Flyable:
    """Mixin for flying capability"""
    def fly(self):
        return f"{self.name} is flying!"
class Swimmable:
    """Mixin for swimming capability"""
    def swim(self):
        return f"{self.name} is swimming!"
class Animal:
    """Base animal class"""
    def __init__(self, name):
        self.name = name
    def eat(self):
        return f"{self.name} is eating"

# Duck inherits from multiple classes
class Duck(Animal, Flyable, Swimmable):
    """Duck can fly and swim"""
    def __init__(self, name):
        super().__init__(name)
    def quack(self):
        return f"{self.name} says Quack!"
```

```
# Fish only swims
class Fish(Animal, Swimmable):
    """Fish can only swim"""
    def __init__(self, name):
        super().__init__(name)

# Create instances
duck = Duck("Donald")
fish = Fish("Nemo")

# Duck has all capabilities Duck
print(duck.eat()) # From Animal
print(duck.fly()) # From Flyable
print(duck.swim()) # From Swimmable
print(duck.quack()) # From Duck
# Fish can only eat and swim
print(fish.eat())
print(fish.swim())
# print(fish.fly()) # Error! Fish can't fly
# Check inheritance
print(isinstance(duck, Animal)) # True
print(isinstance(duck, Flyable)) # True
print(isinstance(duck, Swimmable)) # True
print(isinstance(fish, Flyable)) # False
```

Polymorphism

- Method Overriding

```
class Shape:
    """Base shape class"""
    def __init__(self, name):
        self.name = name
    def area(self):
        """Calculate area - to be overridden"""
        return 0
    def perimeter(self):
        """Calculate perimeter - to be overridden"""
        return 0
    def describe(self):
        return f"This is a {self.name}"

class Rectangle(Shape):
    """Rectangle class"""
    def __init__(self, width, height):
        super().__init__("Rectangle")
        self.width = width
        self.height = height
    def area(self):
        """Override area calculation"""
        return self.width * self.height
    def perimeter(self):
        """Override perimeter calculation"""
        return 2 * (self.width + self.height)
```

```
class Circle(Shape):
    """Circle class"""
    def __init__(self, radius):
        super().__init__("Circle")
        self.radius = radius
    def area(self):
        """Override area calculation"""
        import math
        return math.pi * self.radius ** 2
    def perimeter(self):
        """Override perimeter calculation"""
        import math
        return 2 * math.pi * self.radius
class Triangle(Shape):
    """Triangle class"""
    def __init__(self, base, height, side1, side2):
        super().__init__("Triangle")
        self.base = base
        self.height = height
        self.side1 = side1
        self.side2 = side2
    def area(self):
        """Override area calculation"""
        return 0.5 * self.base * self.height
    def perimeter(self):
        """Override perimeter calculation"""
        return self.base + self.side1 + self.side2
```

```
# Polymorphism in action
shapes = [
    Rectangle(5, 10),
    Circle(7),
    Triangle(6, 8, 5, 5)
]

# Same interface, different implementations
for shape in shapes:
    print(f"\n{shape.describe()}")
    print(f"Area: {shape.area():.2f}")
    print(f"Perimeter: {shape.perimeter():.2f}")
```

Polymorphism

- Duck Typing

```
# "If it walks like a duck and quacks like a duck, it must be a duck"
class Dog:
    def speak(self):
        return "Woof!"
class Cat:
    def speak(self):
        return "Meow!"
class Duck:
    def speak(self):
        return "Quack!"
class Robot:
    def speak(self):
        return "Beep boop!“

# Function that works with any object that has speak()
def make_it_speak(thing):
    """Polymorphic function using duck typing"""
    print(thing.speak())

# All these work!
animals = [Dog(), Cat(), Duck(), Robot()]
for animal in animals:
    make_it_speak(animal)
```

```
# Another example
class File:
    def read(self):
        return "Reading from file..."
class Database:
    def read(self):
        return "Reading from database..."
class API:
    def read(self):
        return "Reading from API..."

def process_data(source):
    """Works with any object that has read()"""
    data = source.read()
    print(f"Processing: {data}")

# Polymorphic usage
sources = [File(), Database(), API()]
for source in sources:
    process_data(source)
```

Encapsulation

- Public, Protected, and Private Members

```
class BankAccount:
    """Bank account demonstrating encapsulation"""
    def __init__(self, account_number, balance):
        # Public attribute
        self.account_number = account_number

        # Protected attribute (convention: single underscore)
        self._balance = balance

        # Private attribute (name mangling: double underscore)
        self.__pin = "1234"

        # Public method
        def get_balance(self):
            """Public getter for balance"""
            return self._balance

        def deposit(self, amount):
            """Public method to deposit money"""
            if amount > 0:
                self._balance += amount
                print(f"Deposited ${amount}. New balance: ${self._balance}")
            else:
                print("Invalid deposit amount!")
```

```
    def withdraw(self, amount, pin):
        """Public method to withdraw money"""
        if not self.__verify_pin(pin):
            print("Invalid PIN!")
            return False
        if amount > self._balance:
            print("Insufficient funds!")
            return False
        self._balance -= amount
        print(f"Withdrew ${amount}. New balance: ${self._balance}")
        return True

    # Private method
    def __verify_pin(self, pin):
        """Private method to verify PIN"""
        return pin == self.__pin

    # Protected method
    def _calculate_interest(self, rate):
        """Protected method for interest calculation"""
        return self._balance * rate
```

```
# Test encapsulation
account = BankAccount("123-456-789", 1000)

# Public access
print(f"Account: {account.account_number}")
# Works
print(f"Balance: ${account.get_balance()}")
# Works

# Protected access (not recommended but possible)
print(f"Direct balance access: ${account._balance}")
# Works but shouldn't

# Private access (name mangling)
# print(account.__pin) # Error! AttributeError
# But can still access with name mangling (not recommended!)
print(f"PIN (name mangled): {account._BankAccount__pin}")
# Use public methods
account.deposit(500)
account.withdraw(200, "1234")
account.withdraw(200, "0000") # Wrong
```

Encapsulation

- Code Comparison
  - Direct Attribute (Not Recommended)

```
class Person:
    def __init__(self, age):
        self.age = age # No validation
person = Person(25)
person.age = -100 # Allowed but invalid!
```

  - Getter/Setter Methods (Verbose)
  - @property (Best Practice)

```
class Person:
    def __init__(self, age):
        self._age = age
    @property
    def age(self):
        return self._age
    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError("Invalid age")
        self._age = value
person = Person(25)
person.age = 26 # Clean and validated!
```

```
class Person:
    def __init__(self, age):
        self._age = age
    def get_age(self):
        return self._age
    def set_age(self, value):
        if value < 0:
            raise ValueError("Invalid age")
        self._age = value

person = Person(25)
person.set_age(26) #  Verbose syntax
```

Encapsulation

- Property Decorators
  - @property = Makes methods look like attributes
  - @name.setter = Adds validation when setting values
  - Computed property = Not stored, calculated dynamically each time
  - Read-only property = Only getter, no setter
  - Underscore prefix _name = Internal variable (convention)

Feature	Direct Attribute	Getter/Setter Methods	@property
Syntax	person.age	person.get_age()	person.age
Setting	person.age = 26	person.set_age(26)	person.age = 26
Validation	None	Yes	Yes
Simplicity	V	X	V
Pythonic	X	X	V
Encapsulation	X	V	V

Encapsulation

- Property Decorators

```python
class Person:
    """Person class using property decorators"""
    def __init__(self, name, age):
        self._name = name
        self._age = age

    # Getter for name
    @property
    def name(self):
        """Get person's name"""
        return self._name

    # Setter for name
    @name.setter
    def name(self, value):
        """Set person's name with validation"""
        if not value:
            raise ValueError("Name cannot be empty!")
        self._name = value

    # Getter for age
    @property
    def age(self):
        """Get person's age"""
        return self._age >= 18
```

```python
    # Setter for age
    @age.setter
    def age(self, value):
        """Set person's age with validation"""
        if not isinstance(value, int):
            raise TypeError("Age must be an integer!")
        if value < 0 or value > 150:
            raise ValueError("Age must be between 0 and 150!")
        self._age = value

    # Computed property
    @property
    def birth_year(self):
        """Calculate birth year"""
        from datetime import datetime
        current_year = datetime.now().year
        return current_year - self._age

    # Read-only property (no setter)
    @property
    def is_adult(self):
        """Check if person is an adult"""
        return self._age >= 18
```

```python
# Use properties
person = Person("Alice", 25)

# Access like attributes, but using methods
print(f"Name: {person.name}") # Calls getter
print(f"Age: {person.age}") # Calls getter

# Set values with validation
person.name = "Alice Smith" # Calls setter
person.age = 26 # Calls setter

# Computed property
print(f"Birth year: {person.birth_year}")

# Read-only property
print(f"Is adult: {person.is_adult}")
# person.is_adult = False # Error! Can't set read-only property

# Validation works
try:
    person.age = -5 # Invalid age
except ValueError as e:
    print(f"Error: {e}")
try:
    person.name = "" # Empty name
except ValueError as e:
    print(f"Error: {e}")
```